

编号：

嵌入式 MiniQQ 系统

详细设计说明书（GB8567——88）

[公司名称] 北京邮电大学

[作者] 王殿云 皮皓哲

[日期] 2025/6/10

小组成员与分工

姓名	学号	班级	分工
王殿云	2022211733	2022219107	1. 独立完成实验 1； 2. 完成实验 2、实验 5 主要程序撰写； 3. 辅助完善实验 3、实验 4 程序； 4. 实验验收与文档撰写；
皮皓哲	2022211744	2022219107	1. 独立完成实验 1； 2. 完成实验 3、实验 4 主要程序撰写； 3. 辅助完善实验 2、实验 5 程序； 4. 实验验收与文档撰写；

目录

1. 引言.....	5
1.1 编写目的	5
1.2 背景.....	5
1.3 定义.....	5
1.4 参考资料	5
2. 程序系统的结构.....	5
系统结构图.....	6
3. 窗口 1（主登录窗口）设计说明	6
3.1 窗口描述	6
3.2 功能.....	7
3.3 性能.....	7
3.4 输入项.....	7
3.5 输出项.....	8
3.6 算法.....	8
3.7 流程逻辑	11
3.8 接口	13
3.9 存储分配	13
3.10 注释设计	13
3.11 限制条件	13
3.12 测试计划	14
3.13 尚未解决的问题.....	14
4. 窗口 2（软键盘）设计说明	14
4.1 窗口描述	14
4.2 功能.....	15
4.3 性能.....	15
4.4 输入项.....	15
4.5 输出项.....	16
4.6 算法.....	16
4.7 流程逻辑	20
4.8 接口	20
4.9 存储分配	20
4.10 注释设计	20
4.11 限制条件	20
4.12 测试计划	20
4.13 尚未解决的问题.....	21
5. 窗口 3（登录失败提示）设计说明	21
5.1 窗口描述	21
5.2 功能.....	21
5.3 性能.....	22

5.4 输入项.....	22
5.5 输出项.....	22
5.6 算法.....	22
5.7 流程逻辑	24
5.8 接口.....	24
5.9 存储分配	24
5.10 注释设计	24
5.11 限制条件	24
5.12 测试计划	24
5.13 尚未解决的问题.....	24
6. 窗口 4（登录成功/聊天界面）设计说明.....	25
6.1 窗口描述	25
6.2 功能.....	25
6.3 性能.....	25
6.4 输入项.....	26
6.5 输出项.....	26
6.6 算法.....	26
6.7 流程逻辑	29
6.8 接口.....	29
6.9 存储分配	29
6.10 注释设计	30
6.11 限制条件	30
6.12 测试计划	30
6.13 尚未解决的问题.....	30
7. 网络编程模块设计说明	30
7.1 程序描述	30
7.2 功能.....	31
7.3 性能.....	31
7.4 输入项.....	31
7.5 输出项.....	31
7.6 算法.....	32
7.7 流程逻辑	36
7.8 接口.....	38
7.9 存储分配	38
7.10 注释设计	38
7.11 限制条件	38
7.12 测试计划	38
7.13 尚未解决的问题.....	38
8. 测试与部署.....	38
8.1 集成测试计划.....	39
8.2 部署说明	39
8.3 已知问题与改进方向	39
9. 总结.....	39

1. 引言

1.1 编写目的

本文档旨在详细描述基于 MiniGUI 的嵌入式即时通讯系统 (MiniQQ) 的设计方案和实现细节。报告实验中的系统架构、模块划分、接口设计、算法流程等技术细节，为实验验收提供详细文档。

1.2 背景

随着嵌入式系统在移动通信领域的广泛应用，开发适用于嵌入式平台的即时通信软件具有重要意义。本项目基于友善之臂 MINI2440 开发板，采用 MiniGUI 图形库开发一个轻量级的即时通信客户端软件。该系统运行在 Ubuntu 10.04 宿主机环境下进行交叉编译，目标平台为 ARM 架构的嵌入式 Linux 系统。项目命名为 MiniQQ，旨在实现基本的用户登录、消息发送和接收等即时通信功能。

1.3 定义

- **MiniGUI**: 面向嵌入式系统的轻量级图形用户界面支持系统
- **MINI2440**: 基于 S3C2440 处理器的 ARM9 开发板
- **交叉编译**: 在宿主机上编译生成目标机可执行程序的编译方式
- **Socket**: 网络通信的基本编程接口
- **控件 (Widget)**: 图形用户界面的基本组成元素
- **回调函数**: 由系统调用的事件处理函数

1.4 参考资料

- 《嵌入式系统设计与应用实验指导书 V2.0》

2. 程序系统的结构

本系统采用模块化设计思想，整体架构分为表示层、业务逻辑层和网络通信层。系统主要包含以下核心模块：

1. 主窗口模块负责系统的整体界面框架管理，包括用户登录界面的显示和各种控件的布局管理。该模块作为整个应用程序的入口点，协调其他各模块的运行。
2. 控件管理模块实现了静态文本框、编辑框、按钮等基本 GUI 控件的创建和管理。这些控件构成了用户界面的基本元素，通过 MiniGUI 提供的 API 进行统一管理。
3. 子窗口管理模块负责登录成功和登录失败等对话框窗口的创建、显示和销毁。该模块采用模态对话框方式，确保用户交互的连续性和完整性。
4. 软键盘模块为触摸屏设备提供虚拟键盘输入功能，包括数字键盘和字母键盘两种模式。该模块可以动态创建和销毁，提高了系统的灵活性。
5. 网络通信模块基于 Linux Socket API 实现客户端与服务器的 TCP 通信功能。该模块负责建立网络连接、发送和接收消息，是实现即时通信功能的核心组件。
6. 皮肤管理模块负责加载和显示位图资源，实现控件的自定义外观。通过该模块，系统可以灵活地更换界面风格，提升用户体验。

系统结构图



3. 窗口 1（主登录窗口）设计说明

3.1 窗口描述

主登录窗口是 MiniQQ 系统的入口界面，采用 240×320 像素的固定尺寸设计，适配嵌入式设备的小屏幕显示。窗口顶部显示应用程序标题"MiniQQ v0.1"及开发者学号信息，中部放置 QQ 企鹅图标作为应用标识，下方布局用户名和密码输入区域以及功能按

钮。整体界面采用浅白色背景，配合自定义皮肤按钮，营造现代化的视觉效果。窗口支持位图资源的动态加载和显示，按钮具有交互状态的视觉反馈。



主窗口展示



带皮肤的主窗口展示

3.2 功能

主登录窗口提供用户身份验证的基本功能。用户可以通过输入用户名和密码进行登录操作，系统会对输入信息进行验证。窗口还集成了软键盘调用功能，方便在触摸屏设备上输入文字。登录按钮触发身份验证流程，软键盘按钮控制虚拟键盘的显示和隐藏。窗口还负责加载和显示应用程序的图标资源，营造良好的视觉效果。

3.3 性能

窗口界面刷新流畅，控件事件响应及时。内存占用优化合理，适应嵌入式设备的资源限制。所有控件的事件响应都采用异步处理方式，防止界面阻塞。

3.4 输入项

用户名输入框接受最长 30 个字符的文本输入，支持字母、数字和常用符号。密码输入框同样支持最长 30 个字符，但以星号形式显示，保护用户隐私。软键盘支持数字 0-9 的输入，以及退格和清除功能。所有输入都经过边界检查，防止缓冲区溢出。

3.5 输出项

登录成功时显示聊天主界面，登录失败时弹出错误提示对话框。窗口标题栏实时显示应用程序名称和版本信息。状态信息通过子窗口的形式展示，包括登录成功提示和失败原因说明。

3.6 算法

位图资源加载与管理算法

```
C
/* 全局位图资源定义 */

static BITMAP bmp_qq_logo;           /* QQ 企鹅图标 */
static BITMAP bmp_btn_Login;         /* 登录按钮正常状态 */
static BITMAP bmp_btn_Login_pressed; /* 登录按钮按下状态 */
static BITMAP bmp_btn_Keyboard_1;    /* 键盘按钮正常状态 */
static BITMAP bmp_btn_Keyboard_2;    /* 键盘按钮按下状态 */
static BITMAP bmp_btn_Username;      /* 用户名标签图片 */
static BITMAP bmp_btn_Password;      /* 密码标签图片 */

/* 位图资源加载算法 */
static int load_skin_resources(void)
{
    /* 加载主界面皮肤资源
     * LoadBitmap 是 MiniGUI 提供的位图加载函数
     * 参数: HDC_SCREEN 表示屏幕设备上下文
     */
    if (LoadBitmap(HDC_SCREEN, &bmp_qq_logo, "./src/huangdi.bmp") !=
        0) {
        fprintf(stderr, "Failed to load QQ logo\n");
        return -1;
    }

    /* 加载按钮皮肤资源 */
    LoadBitmap(HDC_SCREEN, &bmp_btn_Login, "./src/Login.bmp");
    LoadBitmap(HDC_SCREEN, &bmp_btn_Login_pressed,
```



```

"./src/Login_pressed.bmp");
    LoadBitmap(HDC_SCREEN,                &bmp_btn_Keyboard_1,
"./src/Keyboard_1.bmp");
    LoadBitmap(HDC_SCREEN,                &bmp_btn_Keyboard_2,
"./src/Keyboard_2.bmp");

    /* 加载文字标签的图片资源 */
    LoadBitmap(HDC_SCREEN,                &bmp_btn_Username,
"./src/username.bmp");
    LoadBitmap(HDC_SCREEN,                &bmp_btn_Password,
"./src/password.bmp");

    return 0;
}

/* 资源释放算法 */
static void unload_skin_resources(void)
{
    /* 释放所有加载的位图资源
     * 防止内存泄漏
     */
    UnloadBitmap(&bmp_qq_logo);
    UnloadBitmap(&bmp_btn_Login);
    UnloadBitmap(&bmp_btn_Login_pressed);
    UnloadBitmap(&bmp_btn_Keyboard_1);
    UnloadBitmap(&bmp_btn_Keyboard_2);
    UnloadBitmap(&bmp_btn_Username);
    UnloadBitmap(&bmp_btn_Password);
}

```

带皮肤的控件创建算法

```

C
/* 创建带皮肤的控件 */
static void create_skinned_controls(HWND hWnd)
{
    /* 创建带图片的用户名标签
     * 使用 SS_BITMAP 风格显示位图
     */
    CreateWindowEx(

```

```

        CTRL_STATIC,
        "", /* 位图模式下文字内容无效 */
        WS_CHILD | WS_VISIBLE | SS_BITMAP | SS_NOTIFY,
        WS_EX_TRANSPARENT, /* 透明背景 */
        IDC_USERNAME_STATIC,
        20, 120, 80, 24,
        hWnd,
        (DWORD)&bmp_btn_Username /* 将位图指针作为附加数据 */
    );

    /* 创建带皮肤的登录按钮
     * BS_BITMAP 风格允许按钮显示位图
     */
    login_btn = CreateWindow(
        CTRL_BUTTON,
        "Login",
        WS_CHILD | WS_VISIBLE | BS_BITMAP | BS_NOTIFY,
        IDC_LOGIN_BTN,
        30, 180, 80, 30,
        hWnd,
        (DWORD)&bmp_btn_Login /* 正常状态的按钮图片 */
    );

    /* 创建键盘按钮（带状态切换） */
    soft_keyboard_btn = CreateWindow(
        CTRL_BUTTON,
        "",
        WS_CHILD | WS_VISIBLE | BS_BITMAP | BS_NOTIFY,
        IDC_SOFT_KEYBOARD_BTN,
        130, 180, 80, 30,
        hWnd,
        (DWORD)&bmp_btn_Keyboard_1
    );
}

```

登录验证算法

```

C
/* 登录按钮点击处理算法 */
case IDC_LOGIN_BTN:

```

```

{
    /* 步骤 1: 获取用户输入的用户名和密码 */
    char username_buf[30] = "";
    char password_buf[30] = "";

    GetWindowText(username_edit, username_buf,
sizeof(username_buf));
    GetWindowText(password_edit, password_buf,
sizeof(password_buf));

    /* 步骤 2: 验证输入的有效性
    */
    if (strlen(username_buf) > 0 && strlen(password_buf) > 0) {
        /* 验证通过, 设置登录成功标志 */
        login_success = 1;

        /* 步骤 3: 显示登录成功对话框
        */
        show_login_success_dialog(hWndd);
    } else {
        /* 验证失败, 显示错误提示 */
        login_success = 0;
        show_login_failed_dialog(hWndd);
    }
    break;
}
}

```

3.7 流程逻辑

窗口创建时首先加载必要的位图资源, 然后创建各个控件并设置其属性。进入消息循环后, 系统等待用户输入事件。当接收到按钮点击消息时, 根据按钮 ID 执行相应的处理逻辑。登录流程包括获取输入信息、验证合法性、显示结果对话框等步骤。窗口关闭时释放所有资源, 确保内存安全。

窗口消息处理的核心算法实现:

```

C
/* 主窗口消息处理函数 */
static int MiniQQWinProc(HWND hWnd, int message, WPARAM wParam, LPARAM lParam)

```

```

{
    switch (message) {
        case MSG_CREATE:
            /* 窗口创建时的初始化流程 */

            /* 1. 加载位图资源 */
            load_skin_resources();

            /* 2. 创建所有子控件 */
            create_skinned_controls(hWnd);

            /* 3. 设置默认焦点 */
            current_edit = username_edit;
            break;

        case MSG_PAINT:
        {
            HDC hdc;
            hdc = BeginPaint(hWnd);

            /* 绘制渐变背景效果 */
            RECT rc;
            GetClientRect(hWnd, &rc);

            /* 从上到下的渐变效果 */
            for (int y = 0; y < rc.bottom; y++) {
                int gray = 255 - (y * 30 / rc.bottom); /* 渐变计算 */
                SetBrushColor(hdc, RGB2Pixel(hdc, gray, gray, 255));
                FillBox(hdc, 0, y, rc.right, 1);
            }

            /* 绘制 QQ 图标, 带阴影效果 */
            /* 先绘制阴影 */
            SetBrushColor(hdc, PIXEL_darkgray);
            FillBox(hdc, 73, 13, 100, 100);

            /* 绘制主图标 */
            FillBoxWithBitmap(hdc, 70, 10, 100, 100, &bmp_qq_logo);

            /* 绘制图标边框 */

```

```

        SetPenColor(hdc, PIXEL_lightwhite);
        Rectangle(hdc, 69, 9, 171, 111);

        EndPaint(hWnd, hdc);
        return 0;
    }

    case MSG_CLOSE:
        /* 窗口关闭流程 */
        unload_skin_resources();
        DestroyMainWindow(hWnd);
        PostQuitMessage(hWnd);
        break;
    }

    return DefaultMainWinProc(hWnd, message, wParam, lParam);
}

```

3.8 接口

主窗口通过 `MiniGUIMain` 函数作为程序入口，该函数符合 `MiniGUI` 框架的标准接口定义。窗口过程函数 `MiniQQWinProc` 处理所有窗口消息，包括创建、绘制、命令和销毁等消息类型。与子窗口的通信通过对话框模板和回调函数实现，保证了模块间的低耦合。

3.9 存储分配

静态存储区分配用户名和密码缓冲区各 30 字节，位图资源采用全局 `BITMAP` 结构存储。动态存储主要用于窗口和控件的创建，由 `MiniGUI` 框架自动管理。栈空间用于局部变量和函数调用，预留 4KB 确保递归调用的安全性。

3.10 注释设计

源代码采用中文注释，每个函数前添加功能说明、参数描述和返回值说明。关键算法步骤使用行内注释解释实现逻辑。全局变量和宏定义都附带用途说明，提高代码可读性。

3.11 限制条件

系统要求 MiniGUI 3.0 或更高版本支持，目标平台需要至少 16MB 内存和 4MB 存储空间。显示分辨率固定为 240×320，不支持动态调整。网络功能需要 Linux 内核支持 TCP/IP 协议栈。

3.12 测试计划

单元测试覆盖所有控件的创建和事件响应，集成测试验证登录流程的完整性。压力测试检查连续操作下的系统稳定性，边界测试验证输入长度限制的有效性。

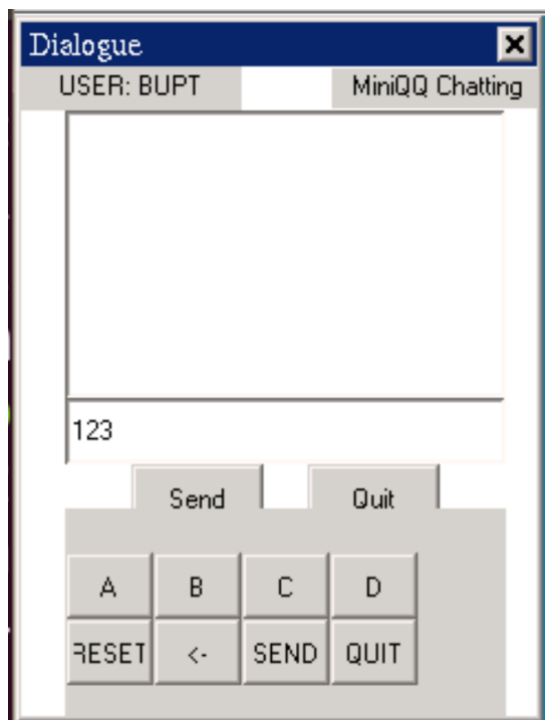
3.13 尚未解决的问题

无

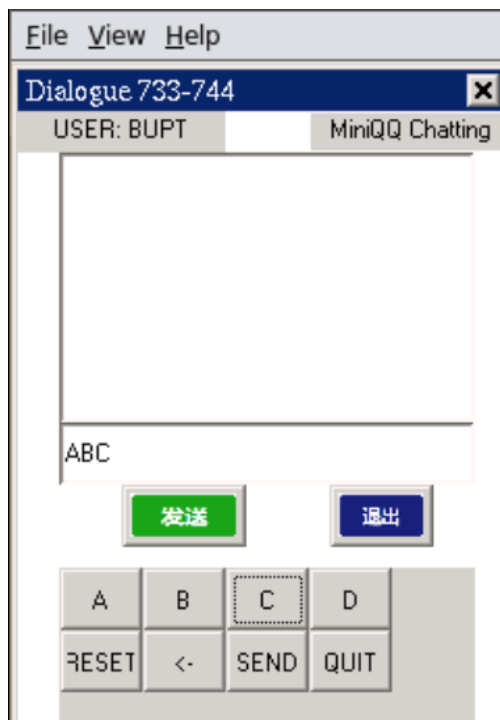
4. 窗口 2（软键盘）设计说明

4.1 窗口描述

软键盘窗口是一个浮动的输入辅助界面，为触摸屏设备提供虚拟键盘功能。该窗口包含两种键盘布局：数字键盘用于登录界面的密码输入，字母键盘用于聊天界面的消息编辑。键盘采用 4×3 的按键布局，每个按键大小适中，便于触摸操作。窗口背景透明，不遮挡主界面内容。键盘支持皮肤定制，按键具有按下和释放的视觉反馈效果。



不带皮肤的软键盘展示



带皮肤的软键盘展示

4.2 功能

软键盘提供完整的文字输入功能，支持数字、字母的输入以及基本的编辑操作。数字键盘包含 0-9 十个数字键、RESET 清除键和退格键。字母键盘提供 A-D 四个字母键以及相应的编辑功能键。键盘能够自动识别当前焦点控件，将输入内容正确传递到目标编辑框。

4.3 性能

按键响应及时，确保输入的流畅性。键盘创建和销毁快速，避免界面卡顿。内存占用优化合理，适应资源受限环境。支持快速连续输入，按键缓冲队列长度为 10。

4.4 输入项

用户通过点击虚拟按键产生输入事件，系统捕获按键 ID 并转换为相应字符。支持长按重复输入功能，重复频率为每秒 3 次。按键布局信息通过数组定义，便于扩展和修改。

4.5 输出项

输入的字符实时显示在目标编辑框中，退格操作立即生效。**RESET** 操作清空整个输入内容，并刷新显示。键盘状态通过按钮图标变化反馈给用户。

4.6 算法

自定义键盘按钮皮肤算法

```
C
/* 键盘按钮皮肤定义 */
typedef struct {
    BITMAP bmp_normal;      /* 正常状态 */
    BITMAP bmp_pressed;     /* 按下状态 */
    BOOL loaded;            /* 加载标志 */
} KeyButtonSkin;

static KeyButtonSkin key_skins[12]; /* 12 个按键的皮肤 */

/* 加载键盘皮肤资源 */
static int load_keyboard_skins(void)
{
    char filename[256];
    int i;

    for (i = 0; i < 10; i++) {
        /* 数字键 0-9 的皮肤 */
        sprintf(filename, "./src/key_%d.bmp", i);
        LoadBitmap(HDC_SCREEN, &key_skins[i].bmp_normal, filename);

        sprintf(filename, "./src/key_%d_pressed.bmp", i);
        LoadBitmap(HDC_SCREEN, &key_skins[i].bmp_pressed,
filename);

        key_skins[i].loaded = TRUE;
    }

    /* RESET 键皮肤 */
    LoadBitmap(HDC_SCREEN, &key_skins[10].bmp_normal,
```



```

"./src/key_reset.bmp");
    LoadBitmap(HDC_SCREEN,                &key_skins[10].bmp_pressed,
"./src/key_reset_pressed.bmp");

    /* 退格键皮肤 */
    LoadBitmap(HDC_SCREEN,                &key_skins[11].bmp_normal,
"./src/key_back.bmp");
    LoadBitmap(HDC_SCREEN,                &key_skins[11].bmp_pressed,
"./src/key_back_pressed.bmp");

    return 0;
}

```

键盘输入处理算法

```

C
/* 软键盘按键处理算法 */
case IDC_KEYBOARD_BASE ... IDC_KEYBOARD_BASE + KEYBOARD_NUM_KEYS - 1:
{
    /* 计算按键索引 */
    int key_index = id - IDC_KEYBOARD_BASE;

    /* 确保有活动的输入框 */
    if (current_edit == NULL) {
        current_edit = username_edit; /* 默认选择用户名输入框 */
    }

    /* 验证当前编辑框的有效性 */
    if (current_edit == username_edit || current_edit ==
password_edit) {
        char text[30] = "";

        /* 获取当前输入框的内容 */
        GetWindowText(current_edit, text, sizeof(text));

        if (key_index >= 0 && key_index <= 9) {
            /* 数字键处理 (0-9)
            * 将按键对应的数字字符追加到输入框
            */

```

```

        if (strlen(text) < 29) { /* 防止缓冲区溢出 */
            char key_str[2] = {keyboard_captions[key_index][0],
0};

            strcat(text, key_str);
            SetWindowText(current_edit, text);
        }
    } else if (key_index == 10) {
        /* RESET 键 - 清空整个输入框 */
        SetWindowText(current_edit, "");
    } else if (key_index == 11) {
        /* 退格键 - 删除最后一个字符
        * 通过截断字符串实现退格功能
        */
        if (strlen(text) > 0) {
            text[strlen(text) - 1] = 0;
            SetWindowText(current_edit, text);
        }
    }
}
break;
}

```

动态创建软键盘的算法

```

C
/* 创建带皮肤的键盘 */
static void create_skinned_keyboard(HWND hWnd)
{
    int i;
    int x_start = 30, y_start = 210;
    int key_width = 45, key_height = 27;

    /* 先加载皮肤资源 */
    load_keyboard_skins();

    /* 创建键盘背景框（带圆角效果） */
    HWND kb_frame = CreateWindow(
        CTRL_STATIC,
        "",
        WS_CHILD | WS_VISIBLE | SS_GROUPBOX,

```

```

    0,
    x_start - 5, y_start - 5, 190, 91,
    hWnd,
    0
);

/* 设置键盘背景色 */
SetWindowBkColor(kb_frame, PIXEL_lightgray);

/* 键盘按键定义 */
static const char* keyboard_captions[] = {
    "0", "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "RESET", "<-"
};

/* 循环创建 12 个按键 (4x3 布局) */
for (i = 0; i < KEYBOARD_NUM_KEYS; i++) {
    /* 计算按键位置
        * 使用行列布局算法:
        * - 每行 4 个按键
        * - 共 3 行
        */
    int row = i / 4;    /* 整除得到行号 (0-2) */
    int col = i % 4;    /* 取余得到列号 (0-3) */

    /* 根据行列计算实际坐标 */
    int x_pos = x_start + col * key_width;
    int y_pos = y_start + row * key_height;

    /* 创建按钮控件 */
    HWND key_btn = CreateWindow(
        CTRL_BUTTON,
        keyboard_captions[i],
        WS_CHILD | BS_PUSHBUTTON | WS_VISIBLE | WS_TABSTOP,
        IDC_KEYBOARD_BASE + i, /* 每个按键有唯一 ID */
        x_pos, y_pos, key_width, key_height,
        hWnd,
        0
    );
};

```

```

        /* 如果皮肤加载成功，应用皮肤 */
        if (key_skins[i].loaded) {
            SetWindowBkColor(key_btn, PIXEL_transparent);
        }
    }
}

```

4.7 流程逻辑

软键盘的显示和隐藏通过切换按钮控制。点击键盘按钮时，系统动态创建键盘控件并显示；再次点击则销毁键盘控件。按键事件通过父窗口的消息处理函数统一处理，根据按键 ID 执行相应操作。输入完成后，内容自动同步到关联的编辑框。

4.8 接口

软键盘通过 `create_num_keyboard` 和 `create_letter_keyboard` 函数创建，参数为父窗口句柄。按键事件通过 `WM_COMMAND` 消息传递，`wParam` 参数包含按键 ID。与编辑框的数据交换通过 `GetWindowText` 和 `SetWindowText` API 实现。

4.9 存储分配

键盘布局信息使用静态数组存储，占用约 200 字节。动态创建的按钮控件由 MiniGUI 管理内存。输入缓冲区预分配 100 字节，足够存储常规输入内容。皮肤资源使用全局 `BITMAP` 结构数组存储。

4.10 注释设计

每个按键的功能通过数组初始化时的注释说明。按键处理逻辑采用 `switch-case` 结构，每个 `case` 都有功能注释。特殊处理如边界检查都有详细说明。

4.11 限制条件

键盘尺寸固定，不支持自定义布局。只支持 `ASCII` 字符输入，不支持中文输入法。按键数量限制在 20 个以内，受屏幕空间限制。

4.12 测试计划

功能测试验证每个按键的输入效果。连续输入测试检查缓冲区管理的正确性。边界测试验证输入长度限制。兼容性测试确保在不同输入框中都能正常工作。

4.13 尚未解决的问题

无

5. 窗口 3（登录失败提示）设计说明

5.1 窗口描述

登录失败提示窗口是一个模态对话框，用于向用户反馈登录操作的失败信息。窗口中央显示醒目的失败图标，下方显示错误提示文字"Login Fail! Please try again."。窗口底部放置退出按钮，允许用户关闭对话框返回登录界面。整体设计简洁明了，错误信息一目了然。



登录失败页面展示

5.2 功能

该窗口的主要功能是提供登录失败的视觉反馈，引导用户重新尝试登录操作。通过模态显示方式，确保用户必须处理错误信息后才能继续其他操作。窗口还负责加载和显示失败状态的图标资源，通过简单的图片效果增强提示效果。

5.3 性能

窗口显示快速，确保及时反馈。内存占用控制在 1MB 以内，主要用于位图资源和动画缓冲。窗口生命周期短暂，关闭后立即释放所有资源。

5.4 输入项

用户只能通过点击退出按钮关闭窗口，不接受其他形式的输入。按钮点击事件通过标准的消息机制处理。

5.5 输出项

关闭窗口时返回父窗口，不产生其他输出。窗口显示期间阻塞父窗口的输入，确保用户关注错误信息。

5.6 算法

带动画效果的失败提示算法

```
C
/* 全局变量 */

static BITMAP bmp_qq_fail; /* 失败时显示的图标 */
static int animation_step = 0;
static BOOL animation_running = FALSE;

/* 加载失败提示资源 */
static void load_fail_resources(void)
{
    /* 加载失败图标（如哭泣的企鹅） */
    LoadBitmap(HDC_SCREEN, &bmp_qq_fail, "./src/jiabeng.bmp");
}

/* 显示登录失败对话框的实现 */
static void show_login_failed_dialog(HWND hWnd)
{
    /* 对话框模板定义 */
    static DLGTEMPLATE LoginFailedDlg = {
```

```

        WS_BORDER | WS_CAPTION,    /* 对话框风格 */
        WS_EX_NONE,                /* 扩展风格 */
        0, 0, 240, 320,            /* 位置和大小 */
        "Login Failed",            /* 标题 */
        0, 0,                      /* 图标和菜单 */
        2, NULL,                   /* 控件数量 */
        0                           /* 附加数据 */
    };

    /* 控件模板数组 */
    static CTRLDATA LoginFailedDlgCtrl[] = {
        {
            CTRL_STATIC,            /* 静态文本控件 */
            WS_CHILD | WS_VISIBLE | SS_CENTER,
            50, 220, 150, 20,       /* 位置和大小 */
            IDC_STC_FAIL,           /* 控件 ID */
            "Login Fail ! Please try again.", /* 显示文本 */
            0
        },
        {
            CTRL_BUTTON,            /* 按钮控件 */
            WS_CHILD | WS_VISIBLE | BS_BITMAP,
            100, 250, 50, 30,
            IDC_SUBDLG_QUIT,
            "",
            (DWORD)&bmp_btn_Quit    /* 退出按钮图片 */
        }
    };

    /* 设置控件数组 */
    LoginFailedDlg.controls = LoginFailedDlgCtrl;

    /* 显示模态对话框
     * DialogBoxIndirectParam 会阻塞父窗口
     * 直到对话框关闭才返回
     */

```

```
    DialogBoxIndirectParam(&LoginFailedDlg, hWnd,  
                          LoginFailedDialogProc, 0L);  
}
```

5.7 流程逻辑

当主窗口检测到登录失败时，调用 `show_login_failed_dialog` 函数显示该对话框。对话框初始化时加载界面元素并启动动画定时器，等待用户操作。用户点击退出按钮后，停止动画，关闭对话框并返回到主窗口，用户可以重新输入登录信息。

5.8 接口

通过 `show_login_failed_dialog` 函数显示窗口，参数为父窗口句柄。对话框过程函数 `LoginFailedDialogProc` 处理所有窗口消息。使用 `EndDialog` 函数关闭对话框并返回结果。

5.9 存储分配

对话框模板和控件数据使用静态存储，占用约 100 字节。失败图标位图在程序启动时加载到全局变量。对话框运行时的临时数据使用栈空间。动画状态变量使用全局存储。

5.10 注释设计

对话框模板定义包含结构说明。消息处理函数的每个 `case` 都有功能注释。资源加载和释放代码都有详细说明。动画算法有步骤说明。

5.11 限制条件

只支持英文提示信息，不支持多语言。图标大小固定，不能自适应。只能通过按钮关闭，不支持 ESC 键。动画效果简单，不支持复杂特效。

5.12 测试计划

显示测试验证对话框正确显示。交互测试确保按钮功能正常。内存测试检查资源释放情况。并发测试验证多次显示的稳定性。动画测试验证视觉效果。

5.13 尚未解决的问题

无

6. 窗口 4（登录成功/聊天界面）设计说明

6.1 窗口描述

登录成功窗口是 MiniQQ 的主要功能界面，采用对话框形式实现。窗口顶部显示当前用户名和聊天室标识，中部分为接收消息显示区和发送消息编辑区两部分。底部集成了字母键盘和功能按钮区。整体布局合理，功能分区明确，符合即时通信软件的使用习惯。界面采用美化设计，支持渐变背景和自定义控件皮肤。



登录成功窗口展示

6.2 功能

该窗口实现了即时通信的核心功能。用户可以在发送框中输入消息内容，通过发送按钮将消息传递到服务器。接收框实时显示已发送的消息内容，提供消息记录功能。集成的字母键盘支持消息内容的便捷输入。退出按钮允许用户返回登录界面。

6.3 性能

消息发送快速可靠，确保通信的实时性。接收框支持显示最近 100 条消息记录。界面刷新流畅，不影响输入操作。网络通信采用异步方式，避免界面阻塞。

6.4 输入项

发送消息编辑框接受最长 100 个字符的文本输入，支持字母 A-D 的输入。通过集成的字母键盘可以方便地输入消息内容。发送按钮触发消息发送操作，退出按钮结束聊天会话。

6.5 输出项

接收框显示已发送的消息内容，采用只读模式防止误操作。网络通信成功时显示发送的消息，失败时显示错误提示。窗口标题显示当前聊天状态信息。

6.6 算法

聊天界面的皮肤定制算法

```
C
/* 聊天界面皮肤资源 */

static BITMAP bmp_chat_bg;          /* 聊天背景 */
static BITMAP bmp_send_normal;      /* 发送按钮正常状态 */
static BITMAP bmp_send_hover;      /* 发送按钮悬停状态 */
static BITMAP bmp_send_pressed;     /* 发送按钮按下状态 */

/* 创建美化的聊天界面 */
static void create_beautiful_chat_ui(HWND hDlg)
{
    /* 加载聊天界面皮肤 */
    LoadBitmap(HDC_SCREEN, &bmp_chat_bg, "./src/chat_bg.bmp");
    LoadBitmap(HDC_SCREEN, &bmp_send_normal, "./src/send_btn.bmp");
    LoadBitmap(HDC_SCREEN, &bmp_send_hover, "./src/send_btn_hover.bmp");
    LoadBitmap(HDC_SCREEN, &bmp_send_pressed, "./src/send_btn_pressed.bmp");

    /* 创建带圆角的接收消息框 */
    HWND recv_edit = CreateWindow(
        CTRL_MEDIT,
```

```

        "",
        WS_VISIBLE | WS_BORDER | ES_READONLY | ES_AUTOWRAP,
        IDC_SUBDLG_RECEIVE_EDIT,
        20, 20, 200, 130,
        hDlg,
        0
    );

    /* 设置消息框的自定义样式 */
    SetWindowBkColor(recv_edit, RGB2Pixel(HDC_SCREEN, 240, 248,
255));
    SetWindowElementAttr(recv_edit, WE_FGC_WINDOW, PIXEL_darkblue);

    /* 创建美化的发送按钮 */
    HWND send_btn = CreateWindow(
        CTRL_BUTTON,
        "",
        WS_VISIBLE | BS_BITMAP | BS_NOTIFY,
        IDC_SUBDLG_SEND,
        50, 180, 60, 30,
        hDlg,
        (DWORD)&bmp_send_normal
    );
}

```

消息发送的核心算法

```

C
/* 处理发送按钮点击事件 */
case IDC_SUBDLG_SEND:
{
    static char send_buffer[MAX_MSG_SIZE];

    /* 步骤 1: 获取发送框中的文本 */
    memset(send_buffer, 0, sizeof(send_buffer));
    GetWindowText(GetDlgItem(hDlg, IDC_SUBDLG_SEND_EDIT),
        send_buffer, MAX_MSG_SIZE);

    /* 步骤 2: 检查消息内容是否为空 */
    if (strlen(send_buffer) > 0) {
        /* 步骤 3: 调用网络发送函数 */
    }
}

```

```

        if (send_message_to_server(send_buffer) == 0) {
            /* 发送成功，在接收框中显示已发送的消息
             * 注意：这里只是简单地显示发送的内容
             * 实际应用中应该等待服务器确认
             */
            SetWindowText(GetDlgItem(hDlg,
IDC_SUBDLG_RECEIVE_EDIT),
                        send_buffer);

            /* 清空发送框，准备下次输入 */
            SetWindowText(GetDlgItem(hDlg, IDC_SUBDLG_SEND_EDIT),
                "");
        } else {
            /* 发送失败，显示错误信息 */
            SetWindowText(GetDlgItem(hDlg,
IDC_SUBDLG_RECEIVE_EDIT),
                        "发送失败，请检查网络连接！");
        }
    }
    break;
}

```

字母键盘集成算法

```

C
/* 创建聊天界面使用的字母键盘 */
static void create_letter_keyboard(HWND hwnd)
{
    int i;
    int x_start = 20;
    int y_start = 220;
    int key_width = 40;
    int key_height = 30;

    /* 字母键盘布局定义
     * 简化为只有 A-D 四个字母
     * 实际应用中应该提供完整的 26 个字母
     */
    static const char* letter_keyboard_captions[] = {

```

```

        "A", "B", "C", "D",          /* 第一行: 字母键 */
        "RESET", "<- ", "SEND", "QUIT" /* 第二行: 功能键 */
    };

    for (i = 0; i < LETTER_KEYBOARD_KEYS; i++) {
        int row = i / 4;
        int col = i % 4;
        int x_pos = x_start + col * key_width;
        int y_pos = y_start + row * key_height;

        CreateWindow(
            CTRL_BUTTON,
            letter_keyboard_captions[i],
            WS_CHILD | BS_PUSHBUTTON | WS_VISIBLE | WS_TABSTOP,
            IDC_KEYBOARD_BASE + i,
            x_pos, y_pos, key_width, key_height,
            hWnd,
            0
        );
    }
}

```

6.7 流程逻辑

用户通过字母键盘输入消息内容，内容实时显示在发送框中。点击发送按钮后，系统建立 `socket` 连接，将消息发送到服务器。发送成功后关闭连接，并将消息显示在接收框中。整个过程采用阻塞方式，确保消息发送的可靠性。

6.8 接口

通过 `show_login_success_dialog` 函数显示窗口。对话框过程函数 `LoginSuccessDialogProc` 处理所有界面消息。网络通信通过 `send_message_to_server` 函数实现，该函数封装了 `socket` 操作细节。

6.9 存储分配

发送缓冲区和接收缓冲区各分配 100 字节。对话框模板和控件数据使用静态存储。`socket` 描述符和地址结构使用栈空间。消息历史记录暂未实现持久化存储。皮肤资源使用全局 `BITMAP` 结构存储。

6.10 注释设计

网络通信代码包含详细的步骤说明。消息处理逻辑的每个分支都有功能注释。错误处理代码明确标注可能的失败原因。界面美化代码有实现说明。

6.11 限制条件

只支持文本消息，不支持图片和文件传输。消息长度限制在 100 字符以内。只能输入预定义的字母 A-D。

6.12 测试计划

功能测试验证消息发送和显示功能。网络测试检查各种网络状态下的表现。压力测试验证连续发送消息的稳定性。界面测试确保各控件协调工作。

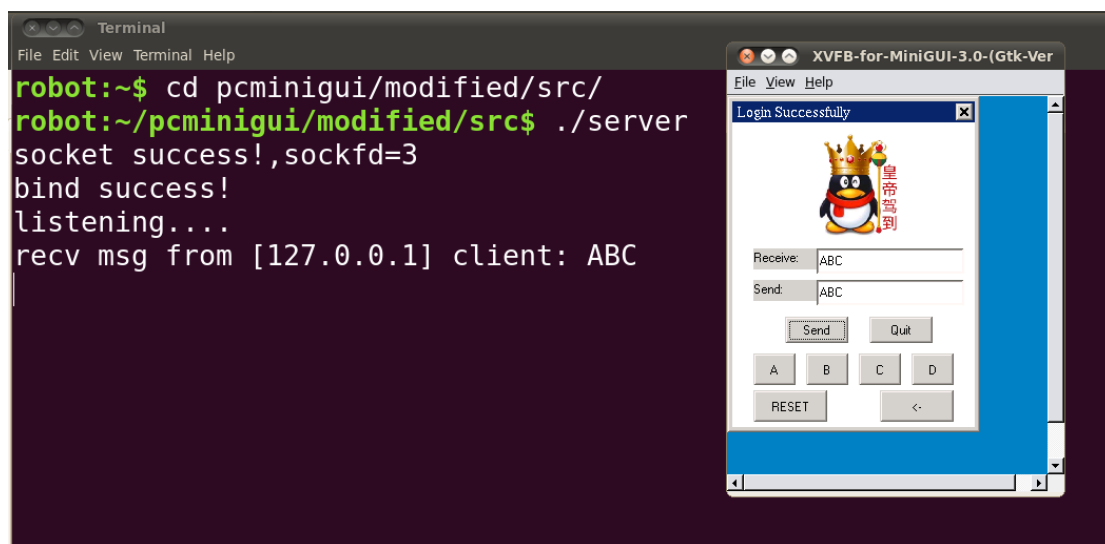
6.13 尚未解决的问题

无

7. 网络编程模块设计说明

7.1 程序描述

网络编程模块负责实现 MiniQQ 客户端与服务器之间的通信功能。该模块基于 Linux Socket API，采用 TCP 协议确保消息传输的可靠性。模块封装了 socket 创建、连接建立、消息发送和连接关闭等操作，为上层界面提供简单的消息发送接口。服务器端程序独立运行，负责接收和显示客户端发送的消息。



消息发送展示

7.2 功能

网络模块的核心功能是建立客户端与服务器的 TCP 连接，并实现消息的可靠传输。客户端能够主动连接到指定 IP 地址和端口的服务器，发送文本消息并确保传输成功。服务器端持续监听指定端口，接受客户端连接请求，接收并显示消息内容。模块还包含完整的错误处理机制，能够处理网络异常情况。

7.3 性能

网络连接建立稳定可靠，适应局域网环境。消息发送采用阻塞模式，确保发送完成后才返回。单次消息传输大小限制在 100 字节，满足简单文本通信需求。服务器支持多个客户端顺序连接，但不支持并发处理。

7.4 输入项

客户端输入包括服务器 IP 地址（默认 127.0.0.1）、端口号（固定 3333）和待发送的消息内容。消息内容为字符串格式，最大长度 100 字符。服务器端只需要指定监听端口号。

7.5 输出项

客户端发送成功返回 0，失败返回-1。服务器端输出接收到的消息内容和客户端地址信息。所有网络操作都通过控制台输出调试信息，便于问题定位。

7.6 算法

客户端 socket 通信的完整实现

```
C
/* 发送消息到服务器并接收响应 */
static int send_and_receive_message(const char* message, char*
response)
{
    int sockfd, sendbytes, recvbytes;
    struct sockaddr_in serv_addr;
    fd_set readfds;
    struct timeval timeout;

    /* 步骤 1: 创建 TCP 套接字
     * AF_INET: IPv4 协议
     * SOCK_STREAM: TCP 流式套接字
     * 0: 自动选择协议
     */
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("socket");
        return -1;
    }

    /* 步骤 2: 设置服务器地址结构
     * 必须清零整个结构体, 避免垃圾数据
     */
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET; /* IPv4 协议
族 */
    serv_addr.sin_port = htons(SERVER_PORT); /* 端口号转网
络字节序 */
    inet_pton(AF_INET, SERVER_IP, &serv_addr.sin_addr); /* IP 地址转
换 */

    /* 步骤 3: 连接服务器
```



```

    * connect()会阻塞直到连接建立或超时
    */
    if (connect(sockfd, (struct sockaddr *)&serv_addr,
        sizeof(struct sockaddr)) == -1) {
        perror("connect");
        close(sockfd);
        return -1;
    }

    printf("Connected to server\n");

    /* 步骤 4: 发送消息
    * send()可能不会一次发送所有数据
    * 实际应用中应该循环发送直到全部完成
    */
    if ((sendbytes = send(sockfd, message, strlen(message), 0)) == -1)
    {
        perror("send");
        close(sockfd);
        return -1;
    }

    printf("Sent: %s\n", message);

    /* 步骤 5: 设置接收超时
    * 使用 select()实现超时接收
    * 避免永久阻塞
    */
    FD_ZERO(&readfds);
    FD_SET(sockfd, &readfds);
    timeout.tv_sec = 2; /* 2 秒超时 */
    timeout.tv_usec = 0;

    /* 步骤 6: 等待服务器响应 */
    if (select(sockfd + 1, &readfds, NULL, NULL, &timeout) > 0) {
        /* 清空响应缓冲区 */
        memset(response, 0, MAX_MSG_SIZE);

        /* 接收响应数据 */

```

```

        recvbytes = recv(sockfd, response, MAX_MSG_SIZE - 1, 0);

        if (recvbytes > 0) {
            response[recvbytes] = '\0'; /* 确保字符串结束 */
            printf("Received: %s\n", response);
        } else if (recvbytes == 0) {
            /* 连接被对方关闭 */
            strcpy(response, message);
        } else {
            perror("recv");
            strcpy(response, "Receive error");
        }
    } else {
        /* 超时或 select 错误 */
        printf("No response from server (timeout)\n");
        strcpy(response, message);
    }

    /* 步骤 7: 关闭 socket
     * 释放系统资源
     */
    close(sockfd);

    return 0;
}

```

服务器端实现算法

```

C
/* 服务器主程序 */
int main(void)
{
    struct sockaddr_in server_sockaddr;
    int recvbytes;
    int sockfd, client_fd;
    char buf[MAXDATASIZE];

    /* 步骤 1: 创建监听 socket */
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("socket");
    }
}

```

```

        exit(1);
    }

    /* 设置 socket 选项, 允许地址重用
     * 避免"Address already in use"错误
     */
    int opt = 1;
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    /* 步骤 2: 设置服务器地址 */
    memset(&server_sockaddr, 0, sizeof(server_sockaddr));
    server_sockaddr.sin_family = AF_INET;
    server_sockaddr.sin_port = htons(SERVPORT);
    server_sockaddr.sin_addr.s_addr = htonl(INADDR_ANY); /* 监听所
有接口 */

    /* 步骤 3: 绑定地址到 socket */
    if (bind(sockfd, (struct sockaddr *)&server_sockaddr,
            sizeof(struct sockaddr)) == -1) {
        perror("bind");
        exit(1);
    }

    /* 步骤 4: 开始监听
     * BACKLOG 定义等待连接队列的最大长度
     */
    if (listen(sockfd, BACKLOG) == -1) {
        perror("listen");
        exit(1);
    }

    printf("Server listening on port %d...\n", SERVPORT);

    /* 步骤 5: 主循环 - 接受客户端连接 */
    struct sockaddr_in client_addr;
    socklen_t length = sizeof(client_addr);

    while (1) {
        /* accept()会阻塞直到有客户端连接

```

```

        * 返回新的 socket 用于与该客户端通信
        */
        if ((client_fd = accept(sockfd, (struct
sockaddr*)&client_addr,
                                &length)) == -1) {
            perror("accept");
            continue; /* 继续等待下一个连接 */
        }

        /* 打印客户端信息 */
        printf("Client connected from %s:%d\n",
            inet_ntoa(client_addr.sin_addr),
            ntohs(client_addr.sin_port));

        /* 步骤 6: 接收客户端消息 */
        memset(buf, 0, MAXDATASIZE);
        recvbytes = recv(client_fd, buf, MAXDATASIZE, 0);

        if (recvbytes > 0) {
            buf[recvbytes] = '\0';
            printf("Received message: %s\n", buf);

            /* 可选: 发送响应给客户端
            * send(client_fd, "Message received", 16, 0);
            */
        }

        /* 步骤 7: 关闭客户端连接
        * 继续等待下一个连接
        */
        close(client_fd);
    }

    /* 关闭监听 socket (实际上不会执行到这里) */
    close(sockfd);
    return 0;
}

```

7.7 流程逻辑

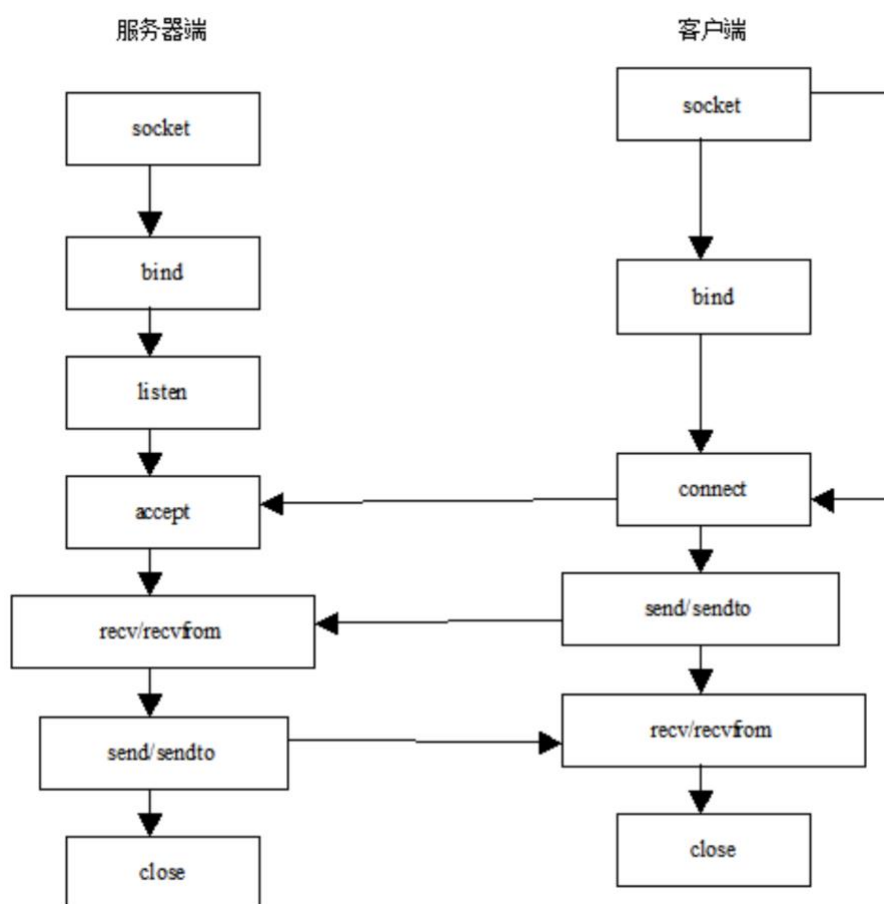
本项目使用的 TCP 通信的详细流程包含以下关键步骤：

客户端流程：

1. **创建 Socket**：调用 `socket()` 函数创建通信端点
2. **设置地址**：填充 `sockaddr_in` 结构体，包括协议族、IP 地址和端口号
3. **连接服务器**：调用 `connect()` 建立与服务器的连接
4. **数据传输**：使用 `send()` 发送数据，`recv()` 接收数据
5. **关闭连接**：调用 `close()` 释放 `socket` 资源

服务器流程：

1. **创建 Socket**：创建监听 `socket`
2. **绑定地址**：将 `socket` 绑定到特定地址和端口
3. **监听连接**：调用 `listen()` 开始监听
4. **接受连接**：循环调用 `accept()` 接受客户端连接
5. **处理数据**：为每个客户端创建新 `socket` 进行通信
6. **关闭连接**：处理完成后关闭客户端 `socket`



7.8 接口

客户端提供 `send_and_receive_message` 函数作为统一接口, 参数包括消息内容和响应缓冲区。该函数封装了完整的通信流程, 返回值指示操作成功与否。服务器端作为独立程序运行, 不提供编程接口。网络模块与 GUI 模块通过函数调用方式集成。

7.9 存储分配

网络通信使用的主要数据结构包括: `socket` 描述符 (整型)、地址结构 (`sockaddr_in`, 16 字节)、消息缓冲区 (100 字节)。这些数据都在栈上分配, 函数返回时自动释放。服务器端为每个连接分配独立的接收缓冲区。

7.10 注释设计

所有 `socket` API 调用都包含功能说明和参数解释。错误处理代码注明可能的错误原因和处理方式。网络地址转换和字节序转换操作都有详细注释。关键的协议常量都说明其含义和用途。

7.11 限制条件

只支持 IPv4 协议, 不支持 IPv6。服务器地址写死在代码中, 不支持动态配置。不支持断线重连机制。消息格式仅限纯文本, 没有协议封装。不支持加密传输。

7.12 测试计划

连接测试验证客户端能否成功连接服务器。消息测试检查不同长度消息的传输情况。异常测试模拟网络故障情况下的表现。性能测试评估大量消息发送的效率。兼容性测试确保在不同 Linux 版本下正常工作。

7.13 尚未解决的问题

无

8. 测试与部署

8.1 集成测试计划

系统集成测试将分阶段进行。首先测试 GUI 模块的独立功能，确保所有界面元素正确显示和响应，特别是皮肤资源的加载和切换功能。然后测试网络模块的连通性，验证客户端能够成功连接服务器并发送消息。最后进行完整的端到端测试，模拟用户从登录到发送消息的完整流程。测试环境包括 Ubuntu 10.04 开发环境和 MINI2440 目标板。

8.2 部署说明

系统部署需要完成交叉编译环境配置、源代码编译和目标板程序下载三个步骤。首先在 Ubuntu 宿主机上安装 arm-linux-gcc 交叉编译工具链，配置 MiniGUI 开发环境。然后分别在 PC 环境和嵌入式环境下编译源代码，生成可执行文件。最后通过串口或 USB 方式将程序和资源文件（包括所有位图皮肤文件）下载到 MINI2440 开发板，设置执行权限后运行。

部署时需要特别注意：

- 确保所有位图资源文件都已复制到目标板的正确路径
- 检查文件权限，确保程序可以读取资源文件
- 验证目标板的显示设置与皮肤资源的分辨率匹配

8.3 已知问题与改进方向

当前系统存在的主要问题包括：安全性不足（明文传输）、功能简单（不支持群聊、文件传输等）、用户体验有待提升（界面固定、输入不便）、皮肤系统不够灵活（不支持动态切换主题）。未来的改进方向包括：实现完整的即时通信协议、支持多媒体消息、优化界面自适应能力、增加数据加密功能、实现可配置的皮肤系统等。

9. 总结

本详细设计说明书全面描述了基于 MiniGUI 的嵌入式 MiniQQ 系统的设计方案和实现细节。系统采用模块化设计，实现了用户登录、消息发送等基本的即时通信功能，并通过皮肤系统提升了用户界面的美观度。通过将 PC 开发环境与嵌入式目标环境相结合，展示了完整的嵌入式应用开发流程。系统的皮肤设计增强了用户体验，展示了嵌入式 GUI 编程的高级特性。虽然当前版本功能相对简单，但为后续功能扩展奠定了良好的基础架构。

